

Creational Design Patterns Assignment (Factory, Abstract Factory, Builder)

1. Objective & Scope

The primary goal of this assignment is to help you understand how multiple creational design patterns interact within a single, cohesive software system. You will design and implement an **Online Examination Subsystem** that mimics real-world enterprise Learning Management Systems (LMS) like Canvas or Moodle.

Through this lab, you will master:

- **Factory Method Pattern:** To decouple the creation of different examination types.
- **Abstract Factory Pattern:** To group families of related question components (data schemas, rendering views, and grading logic).
- **Builder Pattern:** To gracefully construct highly customizable, immutable configuration objects step-by-step without risking constructor bloat.
- **Inversion of Control via Sourcing Strategies:** To dynamically pull test items from simulated question banks or procedural synthetic generators.

2. Problem Statement

The university needs an extensible exam provisioning platform. Instructors require the ability to host a variety of test formats (Practice Quizzes, Midterms, Finals), assign unique configuration presets (time limits, negative grading rules, calculator settings), and source questions dynamically from predefined repositories or automatic pseudo-random text generators.

Crucially, your architecture must honor the **Open/Closed Principle (OCP)**. The core engine should never be modified when a developer wants to add a brand-new exam format (e.g., *Certification Exam*) or a novel question layout (e.g., *Fill-in-the-Blank*).

3. Functional Requirements

3.1 Exam Formats

The platform must handle three main default structures:

- **Practice Quiz:** Unlimited attempts, low stakes, short duration, generally casual rules.
- **Midterm Exam:** Single attempt, strict time limits, comprehensive configurations.
- **Final Exam:** Maximum security configurations, definitive high-stakes weights.

3.2 Decoupled Question Domain

Every question isn't just a simple text string; it is a family of distinct object responsibilities:

1. **Question Entity:** Holds data attributes (e.g., text, point weights).
2. **Renderer Entity:** Formats and displays the question cleanly via UI/Console.
3. **Evaluator Entity:** Evaluates incoming user text answers and applies grading rules.

Supported variations include: Multiple Choice Questions (MCQs), True/False challenges, open-ended Essays, and Programming code challenges.

3.3 Granular Configuration Management

Exams must accept flexible rules customizable through an intuitive builder wizard sequence. These parameters include:

- Title (String), Duration (int minutes), Passing Score (int threshold).
- Behavioral flags: Negative Marking, Question Shuffle, Auto Submit, and Calculator Allowed.

3.4 Question Generation Contexts

Instructors must be able to switch question population modes seamlessly:

- **Manual Mode:** Directly hardcoded or explicit console-prompted setups.
- **Question Bank Mode:** Picks items pseudo-randomly from an pre-populated storage array.
- **Auto/Faker Mode:** Procedurally auto-generates simulated academic content dynamically at runtime.

4. Architectural Design Requirements

Your codebase must strictly follow the architectural pattern guidelines detailed below.

PART A: Factory Method Pattern (Exam Provisioning)

Purpose: Define an interface for creating an object, but let subclasses decide which class to instantiate.

```
interface Exam {  
    String getType();  
    void displayInfo();  
}
```

```
abstract class ExamFactory {  
    public abstract Exam createExam();  
}
```

```
}
```

Your Task: Implement concrete products (PracticeQuiz, MidtermExam, FinalExam) and their corresponding creator factories (PracticeQuizFactory, MidtermExamFactory, FinalExamFactory).

PART B: Abstract Factory Pattern (Question Subsystem)

Purpose: Provide an interface for creating families of related or dependent objects without specifying their concrete classes.

```
interface Question {  
    String getType();  
}  
  
interface QuestionRenderer {  
    void render(Question q);  
}  
  
interface QuestionEvaluator {  
    int evaluate(Question q, String answer);  
}  
  
interface QuestionFactory {  
    Question createQuestion();  
    QuestionRenderer createRenderer();  
    QuestionEvaluator createEvaluator();  
}
```

Your Task: Fully construct separate parallel families for **MCQ**, **True/False**, **Essay**, and **Programming** questions, along with their respective distinct sub-factories (MCQFactory, TrueFalseFactory, etc.).

PART C: Builder Pattern (Exam Configuration)

Purpose: Separate the construction of a complex object from its representation so that the same construction process can create different representations.

```
class ExamConfiguration {  
    String title;
```

```

    int duration;
    int passingScore;
    boolean negativeMarking;
    boolean questionShuffle;
    boolean autoSubmit;
    boolean calculatorAllowed;
}

class ExamBuilder {
    private ExamConfiguration config = new ExamConfiguration();

    .....
}

```

PART D: Question Sourcing Integration

Purpose: Decouple the origin of structural text data using inversion of control interfaces. Concrete Question Factories must accept an abstract QuestionSource via dependency injection.

```

interface QuestionSource {
    Question getQuestion();
}

class BankQuestionSource implements QuestionSource {
    // Fetches random predefined items from a QuestionBank class
}

class FakerQuestionSource implements QuestionSource {
    // Procedurally generates mock question strings via a QuestionFaker context
}

```

5. System Execution Workflow

Your central orchestration driver client file (Main.java) must systematically perform the following step-by-step sequence to verify code integrity:

1. Instantiate a targeted ExamFactory subclass to pick the foundational exam format.
2. Declare an active QuestionSource configuration strategy (e.g., Bank mode or Faker

mode).

3. Initialize Abstract Factories by feeding them the target data source strategy context.
4. Chain fluid API method calls via an ExamBuilder pipeline instance to construct runtime parameters.
5. Generate and log a clean, unified dashboard summary showing system status layout results to the console window.

6. Target Expected Console Output

Your final system report printed on terminal windows should visually approximate the structured dashboard mock format below:

```
=====
                        EXAM CREATED SUCCESSFULLY
=====

Type:      Midterm Exam
Title:     OOP Midterm Exam

Duration:   60 minutes
Passing Score: 50 Marks

Configuration Profiles:
  ✓ Negative Marking Enabled
  ✓ Question Shuffle Activated
  ✗ Embedded Calculator Allowed
  ✓ Auto-Submit On Timeout
Question Sourcing Strategy: Question Bank Mode
Compiled Structural Components:
- [Type: MCQ]      Points: 2  Difficulty: Medium
- [Type: MCQ]      Points: 2  Difficulty: Easy
- [Type: Essay]    Points: 10 Difficulty: Hard
- [Type: Programming] Points: 20 Difficulty: Hard
- [Type: True/False] Points: 1  Difficulty: Easy
=====
```

7. Submission Deliverables

Each student must submit a pdf file (named after the BSSE roll) containing the solution UML class diagram, the publicly accessible source code link (e.g. GitHub), chats made with any AI assistant. Students must be able to explain the solution, failing to do so will result in zero marking.